BACKEND GELİŞTİRME PROJESİ

TASKORA

Görev ve Ekip Yönetim Sistemi
REST API Proje Tanıtım ve Teknik Uygulama Dokümanı

Node.js · Express.js · REST API · Routing · Middleware · CRUD · JSON

Geliştirici: Yiğit Kaan Bilir

Proje türü: REST API / Backend Geliştirme

Repository: github.com/yiqitkaan/Taskora

Sürüm ve tarih: 1.0.0 · 18 Ekim 2026

1. Proje Tanıtımı

Taskora, yazılım ekiplerinin görevlerini merkezi bir servis üzerinden oluşturması, çalışanlara ataması, durum ve öncelik bilgilerini takip etmesi için tasarlanmış bir görev yönetim platformudur. Bu doküman Taskora'nın Node.js ve Express.js ile geliştirilen backend modülünü kapsar.

Backend tamamen REST API mantığında çalışır. Veriler yerel bir JSON dosyasında tutulur. Taskora Web, Taskora Database ve Taskora Mobile bu sürümde backend ile doğrudan entegre değildir.

1.1 Amaç

Projenin amacı; Node.js çalışma ortamı, Express.js sunucusu, routing, middleware, CRUD işlemleri, HTTP durum kodları ve JSON veri yönetimi konularını gerçek hayata yakın tek bir senaryoda uygulamaktır.

1.2 Problem ve çözüm

Bir ekipte görevlerin kime atandığı, hangi aşamada olduğu ve hangi önceliğe sahip olduğu düzenli tutulmadığında iş takibi zorlaşır. Taskora Backend her görev için başlık, açıklama, durum, öncelik, sorumlu, proje ve zaman bilgilerini tek biçimde saklar ve istemcilere beş temel CRUD endpointi sunar.

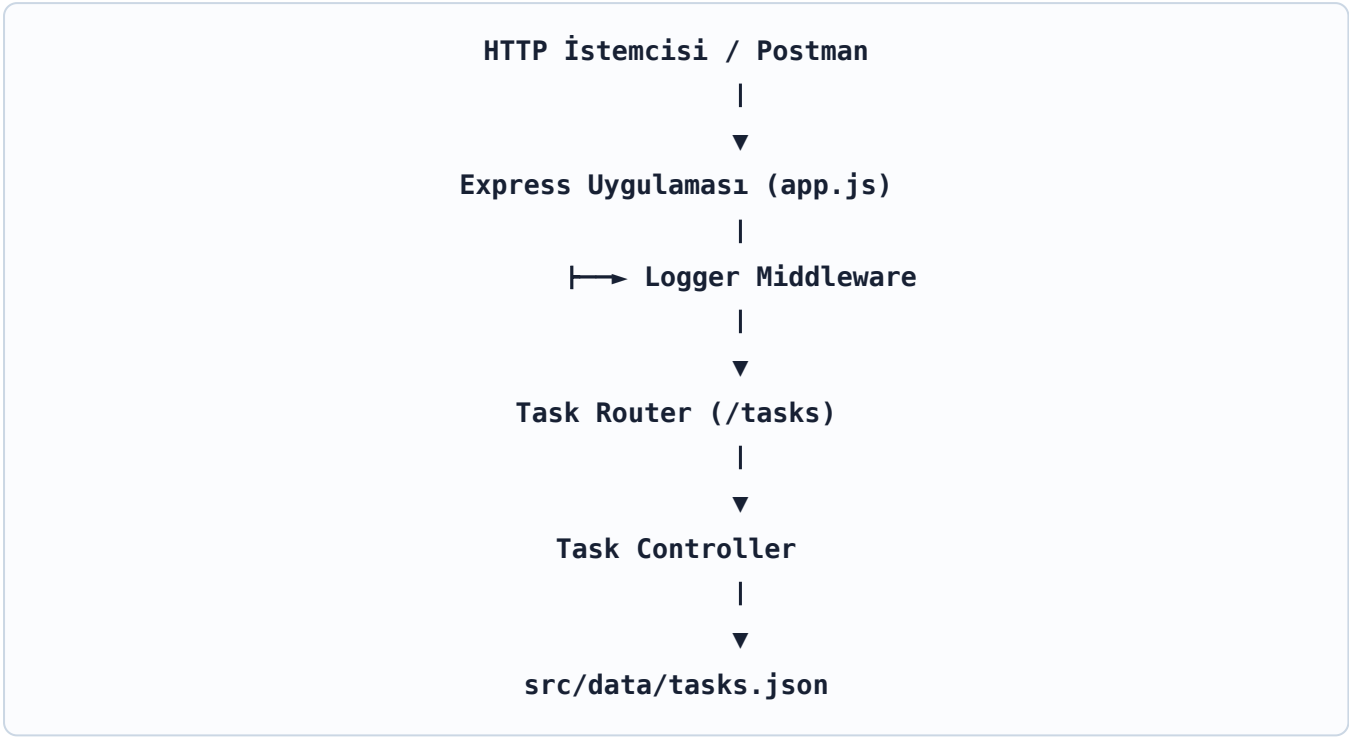
1.3 Proje kapsamı

- Yeni görev oluşturma
- Tüm görevleri listeleme ve isteğe bağlı filtreleme
- ID ile görev detayı görüntüleme
- Görevin seçilen alanlarını güncelleme
- Görev silme
- Tüm istekleri logger middleware ile terminale yazma
- JSON dosyasına kalıcı kayıt ve açıklayıcı hata cevapları

2. Kullanılan Teknolojiler

Teknoloji	Projede kullanım amacı
Node.js 18+	JavaScript backend çalışma ortamı
Express.js 4.21.2	HTTP sunucusu, route ve middleware altyapısı
JavaScript / CommonJS	Controller, route ve uygulama mantığı
JSON	Görev verilerinin yerel ve kalıcı biçimde saklanması
Postman	CRUD endpointlerinin çalıştırılması ve cevapların doğrulanması
Git / GitHub	Versiyon kontrolü ve kaynak kodun yayınlanması

3. Mimari ve İstek Akışı



`app.js` Express uygulamasını kurar, logger ve JSON body middleware'lerini ekler, `/tasks` yolunu task router'a bağlar ve merkezi 404/500 hata cevaplarını tanımlar. Router HTTP methodunu ilgili controller fonksiyonuna yönlendirir. Controller veriyi okuyup doğrular, işlemi gerçekleştirir ve JSON cevap döndürür.

3.1 Klasör yapısı

```
taskora-backend/
├── docs/                          Proje tanıtım dokümanı
├── postman/                       İçe aktarılabilir Postman koleksiyonu
├── screenshots/                   Beş CRUD test ekranı
├── src/
│   ├── controllers/taskController.js
│   ├── data/tasks.json
│   ├── middleware/logger.js
│   ├── routes/taskRoutes.js
│   └── app.js
├── package.json
├── package-lock.json
├── POSTMAN_TESTLERI.md
└── README.md
```

4. Görev Veri Modeli

Alan	Tür	Kural
id	number	Sunucu üretir; pozitif ve benzersizdir
title	string	Zorunlu ve boş olmayan metin
description	string	İsteğe bağlı; varsayılan boş metin
status	string	Zorunlu: pending, in_progress veya completed
priority	string	Zorunlu: low, medium veya high
assignee	string	Zorunlu ve boş olmayan metin
projectId	number/null	İsteğe bağlı; gönderilirse pozitif tam sayı
createdAt	ISO 8601 string	Oluşturma sırasında sunucu yazar
updatedAt	ISO 8601 string	Oluşturma ve güncellemede sunucu yazar

```
{
  "id": 1,
  "title": "Görev API'sini hazırlama",
  "description": "Görev listeleme ve detay endpointleri hazırlanacak.",
  "status": "completed",
  "priority": "high",
  "assignee": "Yiğit Kaan Bilir",
  "projectId": 1,
  "createdAt": "2026-09-19T10:00:00.000Z",
  "updatedAt": "2026-09-19T10:00:00.000Z"
}
```

5. REST API Tasarımı

Method	Endpoint	Başarı	İşlem
POST	/tasks	201	Yeni görev oluşturur
GET	/tasks	200	Tüm görevleri listeler
GET	/tasks/:id	200	Belirli görevi getirir
PUT	/tasks/:id	200	Gönderilen görev alanlarını günceller
DELETE	/tasks/:id	200	Görevi siler

5.1 POST /tasks

`title`, `status`, `priority` ve `assignee` alanları zorunludur. Başarılı işlem yeni görevi, sunucunun ürettiği ID ve zaman alanlarıyla birlikte döndürür.

```
POST /tasks
Content-Type: application/json

{
  "title": "Postman API testi",
  "description": "CRUD isteklerini kontrol et",
  "status": "pending",
  "priority": "high",
  "assignee": "Yiğit Kaan Bilir",
  "projectId": 1
}
```

5.2 GET /tasks ve filtreleme

Temel istek bütün görevleri JSON dizisi olarak döndürür. İsteğe bağlı ileri seviye özellik olarak durum ve öncelik filtreleri birlikte veya ayrı kullanılabilir.

```
GET /tasks
GET /tasks?status=completed
GET /tasks?priority=high
GET /tasks?status=in_progress&priority=medium
```

5.3 GET /tasks/:id

Pozitif tam sayı biçimindeki ID ile tek görev döndürülür. Geçersiz ID biçimi `400`, bulunmayan görev `404` üretir.

5.4 PUT /tasks/:id

İstemci yalnızca değiştirmek istediği alanları gönderir. `id`, `createdAt` ve `updatedAt` istemci tarafından değiştirilemez. Başarılı güncellemede `updatedAt` yenilenir.

```
{
  "status": "in_progress",
  "priority": "medium"
}
```

5.5 DELETE /tasks/:id

Görev JSON dizisinden çıkarılır ve veri dosyası güncellenir.

```
{ "message": "Görev başarıyla silindi." }
```

6. Middleware ve Hata Yönetimi

6.1 Logger middleware

`src/middleware/logger.js` bütün isteklerden önce çalışır. ISO zaman bilgisini okunabilir biçime getirerek method ve orijinal URL ile terminale yazar.

```
[2026-10-18 20:33:18] POST /tasks
[2026-10-18 20:33:19] GET /tasks
[2026-10-18 20:33:20] PUT /tasks/4
```

6.2 Doğrulamalar

- İstek gövdesi JSON nesnesi olmalıdır.
- Oluşturmada zorunlu alanlar bulunmalıdır.
- Başlık ve sorumlu boş metin olamaz.
- Durum ve öncelik yalnızca tanımlı değerlerden biri olabilir.
- projectId gönderilirse pozitif tam sayı olmalıdır.
- Güncellemede en az bir düzenlenebilir alan bulunmalıdır.
- Tanımlanmayan veya sunucuya ait alanlar değiştirilemez.

6.3 HTTP durum kodları

Kod	Kullanım	Örnek cevap
200 OK	Listeleme, detay, güncelleme ve silme başarılı	Görev veya başarı mesajı
201 Created	Görev oluşturma başarılı	Oluşturulan görev
400 Bad Request	Geçersiz ID, alan veya JSON gövdesi	{ "error": "..." }
404 Not Found	Görev ya da endpoint bulunamadı	{ "error": "Görev bulunamadı." }
500 Internal Server Error	Beklenmeyen dosya/sunucu hatası	{ "error": "Sunucu hatası." }

7. JSON Veri Yönetimi

Controller her işlemde `src/data/tasks.json` dosyasını UTF-8 olarak okur ve içeriğin dizi olduğunu doğrular. Yazma işlemi önce süreç ID'sini içeren geçici bir dosyaya yapılır, ardından dosya yeniden adlandırılarak asıl veri dosyasının yerini alır. Böylece yarım yazılmış JSON bırakma riski azaltılır.

Yeni ID, mevcut en yüksek ID'nin bir fazlasıdır. Oluşturma, güncelleme ve silme işlemleri dosyaya yazıldığı için sunucu kapatılıp açıldığında görev değişiklikleri korunur.

> Bu sürümde veritabanı veya authentication bulunmaz. JSON dosyası yerel veri katmanı olarak kullanılır. TaskoraDB entegrasyonu sonraki geliştirme aşamasıdır.

8. Test Planı ve Doğrulama

8.1 Temel Postman akışı

1. **POST /tasks:** Görev oluşturulur; `201 Created`.
2. **GET /tasks:** Yeni kayıt listede görülür; `200 OK`.
3. **GET /tasks/:id:** Doğru görev döner; `200 OK`.
4. **PUT /tasks/:id:** Durum ve öncelik değişir; `200 OK`.
5. **DELETE /tasks/:id:** Görev silinir; `200 OK`.

Hazır Postman koleksiyonu ilk cevaptaki ID'yi koleksiyon değişkenine kaydeder. Böylece detay, güncelleme ve silme istekleri aynı kaydı kullanır. Her istekte method, URL, HTTP status ve response JSON görünür şekilde doğrulanır.

8.2 Gerçek çalışma doğrulaması

18 Ekim 2026 tarihinde temiz bir geçici görevle uçtan uca HTTP testi uygulanmış ve başlangıç JSON'u test sonunda geri yüklenmiştir.

Kontrol	Beklenen	Sonuç
Görev oluşturma	201 ve yeni görev	**BAŞARILI**
Listeleme / detay / filtre	200 ve doğru veri	**BAŞARILI**
Güncelleme	200 ve güncel alanlar	**BAŞARILI**
Silme	200 ve başarı mesajı	**BAŞARILI**
Silme sonrası detay	404	**BAŞARILI**
Eksik alan / bozuk JSON	400	**BAŞARILI**
Logger	Method, URL ve zaman	**BAŞARILI**
npm güvenlik denetimi	0 açık	**BAŞARILI**

9. Kurulum Özeti

```
git clone https://github.com/yiqitkaan/Taskora.git
cd Taskora/taskora-backend
npm ci
npm start
```

Varsayılan adres `http://localhost:3000`'dir. Alternatif port için `PORT=4000 npm start` kullanılabilir.

10. Teknik Özellik Matrisi

Teknik özellik	Taskora karşılığı	Durum
Node.js projesi ve Express.js	package.json ve Express 4 uygulaması	**Tamamlandı**
Klasör ve routing yapısı	controllers, routes, middleware, data	**Tamamlandı**
Görev ekleme	POST /tasks	**Tamamlandı**
Görev listeleme	GET /tasks	**Tamamlandı**
Görev detayı	GET /tasks/:id	**Tamamlandı**
Görev güncelleme	PUT /tasks/:id	**Tamamlandı**
Görev silme	DELETE /tasks/:id	**Tamamlandı**
Temel logger middleware	Method, endpoint ve zaman damgası	**Tamamlandı**
JSON veri yönetimi	tasks.json okuma ve atomik yazma	**Tamamlandı**
README kurulum notu	Ayrıntılı kurulum, endpoint ve test rehberi	**Tamamlandı**
Proje tanıtım dokümanı	Bu PDF ve düzenlenebilir HTML kaynağı	**Tamamlandı**
Postman temel CRUD testi	Koleksiyon ve adım adım test rehberi	**Hazır**
Postman ekran görüntüleri	screenshots klasöründe beş gerçek çıktı	**Tamamlandı**
Public GitHub repository	github.com/yiqitkaan/Taskora	**Tamamlandı**

11. İsteğe Bağlı Özellikler

Durum ve öncelik filtreleri uygulanmıştır. Görev alanlarının doğrulanması ve açıklayıcı 400 cevapları da controller seviyesinde eklenmiştir. Arama, assignee filtresi, sayfalama, sıralama ve raporlama endpointleri mevcut sürümde bulunmaz.

12. Sonuç

Taskora Backend; çalışan REST API, eksiksiz görev CRUD operasyonları, ayrık routing/controller yapısı, logger middleware ve JSON veri yönetimi sunar. Kurulum README'si, proje tanıtım dokümanı, Postman koleksiyonu ve test rehberiyle doğrulama süreci tekrarlanabilir hale getirilmiştir.

> **Proje özeti:** Kaynak kod + package.json/package-lock.json + README + proje tanıtım PDF'i + Postman koleksiyonu ve gerçek test görüntüleri.

Hazırlayan: Yiğit Kaan Bilir · 2026